Autonomyx Optimize — Next.js Codebase Files (Batch 3)

This batch adds the remaining admin and governance surfaces, detail routes, mutation hooks, and reusable loading/error wrappers.

Included in this batch: - `src/app/(dashboard)/policy-builder/page.tsx` - `src/app/(dashboard)/roles/page.tsx` - `src/app/(dashboard)/settings/page.tsx` - `src/app/(dashboard)/approvals/[id]/page.tsx` - `src/app/(dashboard)/executions/[id]/page.tsx` - `src/components/shared/loading-state.tsx` - `src/components/shared/error-state.tsx` - `src/components/app-shell/tenant-switcher.tsx` - `src/components/app-shell/role-preset-switcher.tsx` - `src/features/approvals/mutations.ts` - `src/features/executions/mutations.ts` - `src/lib/mock/repositories/policy-rules-repo.ts` - `src/lib/mock/repositories/settings-repo.ts` - `src/lib/mock/repositories/role-views-repo.ts` - `src/features/policy-builder/queries.ts` - `src/features/settings/queries.ts` - `src/features/roles/queries.ts` - `src/types/settings.ts`

`src/types/settings.ts`

```ts
export interface PolicyRule {
  id: string;
  name: string;
  mode: string;
  applies: string;
  condition: string;
  execution: string;
  status: string;
}

export interface RoleView {
  id: string;
  title: string;
  focus: string;
  widgets: string[];
}

export interface IntegrationStatus {
  name: string;
  status: string;
}

export interface SettingsSnapshot {
```

```
  workspaceName: string;
  defaultCurrency: string;
  approvalTimeoutHours: number;
  defaultExecutionMode: string;
  integrations: IntegrationStatus[];
  notifications: IntegrationStatus[];
  agentConfiguration: IntegrationStatus[];
}
```

src/lib/mock/repositories/policy-rules-repo.ts

```
import type { ApiResult } from "@/lib/api/response";
import type { PolicyRule } from "@/types/settings";

const policyRules: PolicyRule[] = [
  {
    id: "pol-1",
    name: "Low-risk SaaS cleanup",
    mode: "Auto-approve",
    applies: "SaaS · inactive_licenses",
    condition: "Risk low and savings < $1K",
    execution: "Controlled live run",
    status: "Enabled",
  },
  {
    id: "pol-2",
    name: "Production cloud changes",
    mode: "Require approval",
    applies: "Cloud · production",
    condition: "Any production impact",
    execution: "Manual after approval",
    status: "Enabled",
  },
  {
    id: "pol-3",
    name: "Contract renegotiation threshold",
    mode: "Draft only",
    applies: "Vendor · contract_renegotiation",
    condition: "Annual commit >= $50K",
    execution: "Draft memo only",
    status: "Enabled",
  },
];

export async function listPolicyRules(): Promise<ApiResult<PolicyRule[]>> {
  return { ok: true, data: policyRules };
```

```
}

export async function getPolicyRuleById(id: string):
Promise<ApiResult<PolicyRule>> {
  const item = policyRules.find((rule) => rule.id === id);
  if (!item) return { ok: false, error: "Policy rule not found" };
  return { ok: true, data: item };
}
```

src/lib/mock/repositories/settings-repo.ts

```
import type { ApiResult } from "@/lib/api/response";
import type { SettingsSnapshot } from "@/types/settings";

const snapshot: SettingsSnapshot = {
  workspaceName: "Autonomyx Internal",
  defaultCurrency: "USD",
  approvalTimeoutHours: 48,
  defaultExecutionMode: "Controlled live run",
  integrations: [
    { name: "AWS Billing", status: "Connected" },
    { name: "Okta", status: "Connected" },
    { name: "OpenAI Gateway", status: "Connected" },
    { name: "Procurement System", status: "Pending" },
  ],
  notifications: [
    { name: "Email alerts", status: "Enabled" },
    { name: "Slack approvals", status: "Enabled" },
    { name: "Weekly digest", status: "Enabled" },
  ],
  agentConfiguration: [
    { name: "Skeptic agent", status: "Required" },
    { name: "Execution sandbox", status: "Enabled" },
    { name: "Auto-approval rules", status: "12 active" },
  ],
};

export async function getSettingsSnapshot():
Promise<ApiResult<SettingsSnapshot>> {
  return { ok: true, data: snapshot };
}
```

src/lib/mock/repositories/role-views-repo.ts

```
import type { ApiResult } from "@/lib/api/response";
import type { RoleView } from "@/types/settings";

const roleViews: RoleView[] = [
  {
    id: "cfo",
    title: "CFO View",
    focus: "Budget leakage, realized savings, renewal exposure, ROI.",
    widgets: ["Savings funnel", "Top vendor exposure", "Realized vs projected
savings", "Budget risk map"],
  },
  {
    id: "procurement",
    title: "Procurement View",
    focus: "Renewals, negotiation leverage, contract risk, price benchmarks.",
    widgets: ["Upcoming renewals", "Negotiation workspace", "Auto-renew risk",
"Vendor concentration"],
  },
  {
    id: "engineering",
    title: "Engineering View",
    focus: "Cloud waste, LLM efficiency, execution safety, rollback readiness.",
    widgets: ["Idle resources", "Runtime efficiency", "Execution queue",
"Rollback health"],
  },
];

export async function listRoleViews(): Promise<ApiResult<RoleView[]>> {
  return { ok: true, data: roleViews };
}
```

src/features/policy-builder/queries.ts

```
import { useQuery } from "@tanstack/react-query";
import { listPolicyRules } from "@/lib/mock/repositories/policy-rules-repo";

export function usePolicyRulesQuery() {
  return useQuery({
    queryKey: ["policy-rules"],
    queryFn: listPolicyRules,
  });
}
```

src/features/settings/queries.ts

```
import { useQuery } from "@tanstack/react-query";
import { getSettingsSnapshot } from "@/lib/mock/repositories/settings-repo";

export function useSettingsSnapshotQuery() {
  return useQuery({
    queryKey: ["settings-snapshot"],
    queryFn: getSettingsSnapshot,
  });
}
```

src/features/roles/queries.ts

```
import { useQuery } from "@tanstack/react-query";
import { listRoleViews } from "@/lib/mock/repositories/role-views-repo";

export function useRoleViewsQuery() {
  return useQuery({
    queryKey: ["role-views"],
    queryFn: listRoleViews,
  });
}
```

src/features/approvals/mutations.ts

```
import { useMutation, useQueryClient } from "@tanstack/react-query";

async function resolveApproval(input: { id: string; decision: "approved" |
"rejected" | "deferred" | "escalated"; comment?: string }) {
  return {
    ok: true,
    data: input,
  };
}

export function useResolveApprovalMutation() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: resolveApproval,
    onSuccess: async () => {
      await queryClient.invalidateQueries({ queryKey: ["approvals"] });
```

```
      await queryClient.invalidateQueries({ queryKey: ["recommendations"] });
    },
  });
}
```

src/features/executions/mutations.ts

```
import { useMutation, useQueryClient } from "@tanstack/react-query";

async function updateExecution(input: { id: string; action: "pause" |
"rollback" | "resume" }) {
  return {
    ok: true,
    data: input,
  };
}

export function useExecutionActionMutation() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: updateExecution,
    onSuccess: async () => {
      await queryClient.invalidateQueries({ queryKey: ["executions"] });
    },
  });
}
```

src/components/shared/loading-state.tsx

```
import { Loader2 } from "lucide-react";

export function LoadingState({ label = "Loading..." }: { label?: string }) {
  return (
    <div className="flex min-h-[240px] items-center justify-center rounded-3xl
border bg-white p-10">
      <div className="flex items-center gap-3 text-sm text-slate-500">
        <Loader2 className="h-4 w-4 animate-spin" />
        {label}
      </div>
    </div>
  );
}
```

src/components/shared/error-state.tsx

```tsx
import { AlertTriangle } from "lucide-react";

export function ErrorState({
  title = "Something went wrong",
  description = "Please retry or contact support if the problem continues.",
}: {
  title?: string;
  description?: string;
}) {
  return (
    <div className="rounded-3xl border bg-white p-10 text-center">
      <div className="mx-auto mb-4 flex h-12 w-12 items-center justify-center
rounded-full bg-slate-100">
        <AlertTriangle className="h-5 w-5" />
      </div>
      <div className="text-lg font-semibold">{title}</div>
      <div className="mt-2 text-sm text-slate-500">{description}</div>
    </div>
  );
}
```

src/components/app-shell/tenant-switcher.tsx

```tsx
"use client";

import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from
"@/components/ui/select";

interface TenantOption {
  id: string;
  name: string;
  plan: string;
}

export function TenantSwitcher({
  value,
  onChange,
  options,
}: {
  value: string;
  onChange: (value: string) => void;
  options: TenantOption[];
}) {
```

```
  return (
    <Select value={value} onValueChange={onChange}>
      <SelectTrigger className="rounded-2xl">
        <SelectValue placeholder="Select tenant" />
      </SelectTrigger>
      <SelectContent>
        {options.map((tenant) => (
          <SelectItem key={tenant.id} value={tenant.id}>
            {tenant.name} · {tenant.plan}
          </SelectItem>
        ))}
      </SelectContent>
    </Select>
  );
}
```

src/components/app-shell/role-preset-switcher.tsx

```
"use client";

import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from
"@/components/ui/select";

export type RolePreset = "default" | "admin" | "finance" | "procurement" |
"engineering";

export function RolePresetSwitcher({
  value,
  onChange,
}: {
  value: RolePreset;
  onChange: (value: RolePreset) => void;
}) {
  return (
    <Select value={value} onValueChange={(v) => onChange(v as RolePreset)}>
      <SelectTrigger className="min-w-[190px] rounded-2xl">
        <SelectValue placeholder="Role preset" />
      </SelectTrigger>
      <SelectContent>
        <SelectItem value="default">Default</SelectItem>
        <SelectItem value="admin">Admin</SelectItem>
        <SelectItem value="finance">Finance</SelectItem>
        <SelectItem value="procurement">Procurement</SelectItem>
        <SelectItem value="engineering">Engineering</SelectItem>
      </SelectContent>
    </Select>
```

```
  );
}
```

src/app/(dashboard)/policy-builder/page.tsx

```
import { PageHeader } from "@/components/shared/page-header";
import { requirePermission } from "@/lib/auth/guards";
import { listPolicyRules } from "@/lib/mock/repositories/policy-rules-repo";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Input } from "@/components/ui/input";
import { Textarea } from "@/components/ui/textarea";
import { Button } from "@/components/ui/button";
import { Badge } from "@/components/ui/badge";

export default async function PolicyBuilderPage() {
  await requirePermission("editPolicies");
  const result = await listPolicyRules();
  const rule = result.ok ? result.data[0] : null;

  return (
    <div className="grid gap-6 p-6 xl:grid-cols-[340px_1fr]">
      <Card className="rounded-3xl border-0 shadow-sm">
        <CardHeader>
          <CardTitle>Policy Rules</CardTitle>
        </CardHeader>
        <CardContent className="space-y-3">
          {result.ok && result.data.map((item) => (
            <div key={item.id} className="rounded-2xl border p-4">
              <div className="font-medium">{item.name}</div>
              <div className="mt-1 text-xs text-slate-500">{item.applies}</div>
            </div>
          ))}
        </CardContent>
      </Card>

      {rule ? (
        <Card className="rounded-3xl border-0 shadow-sm">
          <CardHeader>
            <div className="flex items-center justify-between gap-4">
              <div>
                <PageHeader title={rule.name} description="Edit approval logic
and execution mode." />
              </div>
              <Badge variant="outline">{rule.status}</Badge>
            </div>
          </CardHeader>
```

```
          <CardContent className="space-y-6">
            <div className="grid gap-4 md:grid-cols-2">
              <Input defaultValue={rule.mode} />
              <Input defaultValue={rule.execution} />
            </div>
            <Input defaultValue={rule.applies} />
            <Textarea defaultValue={rule.condition} className="min-h-[140px]" />
            <pre className="overflow-auto rounded-2xl border bg-slate-50 p-4
text-xs text-slate-700">{`policy_id: ${rule.id}
version: 1
enabled: true
applies_to: ${rule.applies}
conditions: ${rule.condition}
decision:
  mode: ${rule.mode}
  execution_mode: ${rule.execution}`}</pre>
            <div className="flex gap-3">
              <Button>Save Rule</Button>
              <Button variant="outline">Clone</Button>
              <Button variant="outline">Test Policy</Button>
            </div>
          </CardContent>
        </Card>
      ) : null}
    </div>
  );
}
```

src/app/(dashboard)/roles/page.tsx

```
import { PageHeader } from "@/components/shared/page-header";
import { requirePermission } from "@/lib/auth/guards";
import { listRoleViews } from "@/lib/mock/repositories/role-views-repo";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";

export default async function RolesPage() {
  await requirePermission("viewControlTower");
  const result = await listRoleViews();
  const selected = result.ok ? result.data[0] : null;

  return (
    <div className="grid gap-6 p-6 xl:grid-cols-[320px_1fr]">
      <Card className="rounded-3xl border-0 shadow-sm">
        <CardHeader>
          <CardTitle>Role Presets</CardTitle>
        </CardHeader>
```

```
        <CardContent className="space-y-3">
          {result.ok && result.data.map((role) => (
            <div key={role.id} className="rounded-2xl border p-4">
              <div className="font-medium">{role.title}</div>
            </div>
          ))}
        </CardContent>
      </Card>

      {selected ? (
        <div className="space-y-6">
          <PageHeader title={selected.title} description={selected.focus} />
          <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-4">
            {selected.widgets.map((widget) => (
              <Card key={widget} className="rounded-3xl border-0 shadow-sm">
                <CardContent className="p-5">
                  <div className="text-sm font-medium">{widget}</div>
                  <div className="mt-2 text-xs text-slate-500">Optimized for
stakeholder workflow emphasis.</div>
                </CardContent>
              </Card>
            ))}
          </div>
        </div>
      ) : null}
    </div>
  );
}
```

src/app/(dashboard)/settings/page.tsx

```
import { PageHeader } from "@/components/shared/page-header";
import { requirePermission } from "@/lib/auth/guards";
import { getSettingsSnapshot } from "@/lib/mock/repositories/settings-repo";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Input } from "@/components/ui/input";

export default async function SettingsPage() {
  await requirePermission("manageSettings");
  const result = await getSettingsSnapshot();
  if (!result.ok) return null;

  const settings = result.data;

  return (
    <div className="space-y-6 p-6">
```

```
      <PageHeader title="Settings" description="Manage tenant configuration,
integrations, notifications, and agent posture." />

      <div className="grid gap-6 xl:grid-cols-3">
        <Card className="rounded-3xl border-0 shadow-sm xl:col-span-2">
          <CardHeader>
            <CardTitle>Tenant Settings</CardTitle>
          </CardHeader>
          <CardContent className="grid gap-6 md:grid-cols-2">
            <Input defaultValue={settings.workspaceName} />
            <Input defaultValue={settings.defaultCurrency} />
            <Input defaultValue={String(settings.approvalTimeoutHours)} />
            <Input defaultValue={settings.defaultExecutionMode} />
          </CardContent>
        </Card>

        <Card className="rounded-3xl border-0 shadow-sm">
          <CardHeader>
            <CardTitle>Security Posture</CardTitle>
          </CardHeader>
          <CardContent className="space-y-3 text-sm text-slate-600">
            <div>RBAC enforced</div>
            <div>SoD workflow enabled</div>
            <div>Audit retention: 365 days</div>
          </CardContent>
        </Card>
      </div>

      <div className="grid gap-6 xl:grid-cols-3">
        <Card className="rounded-3xl border-0 shadow-sm">
          <CardHeader><CardTitle>Integrations</CardTitle></CardHeader>
          <CardContent className="space-y-3">
            {settings.integrations.map((item) => <div key={item.name}
className="flex items-center justify-between rounded-2xl border
p-4"><span>{item.name}</span><span>{item.status}</span></div>)}
          </CardContent>
        </Card>
        <Card className="rounded-3xl border-0 shadow-sm">
          <CardHeader><CardTitle>Notifications</CardTitle></CardHeader>
          <CardContent className="space-y-3">
            {settings.notifications.map((item) => <div key={item.name}
className="flex items-center justify-between rounded-2xl border
p-4"><span>{item.name}</span><span>{item.status}</span></div>)}
          </CardContent>
        </Card>
        <Card className="rounded-3xl border-0 shadow-sm">
          <CardHeader><CardTitle>Agent Configuration</CardTitle></CardHeader>
          <CardContent className="space-y-3">
```

```
            {settings.agentConfiguration.map((item) => <div key={item.name}
className="flex items-center justify-between rounded-2xl border
p-4"><span>{item.name}</span><span>{item.status}</span></div>)}
          </CardContent>
        </Card>
      </div>
    </div>
  );
}
```

src/app/(dashboard)/approvals/[id]/page.tsx

```
import { notFound } from "next/navigation";
import { PageHeader } from "@/components/shared/page-header";
import { requirePermission } from "@/lib/auth/guards";
import { listApprovals } from "@/lib/mock/repositories/approvals-repo";
import { getRecommendationById } from "@/lib/mock/repositories/recommendations-
repo";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Textarea } from "@/components/ui/textarea";

export default async function ApprovalDetailPage({ params }: { params:
Promise<{ id: string }> }) {
  await requirePermission("approveActions");
  const { id } = await params;

  const approvalsResult = await listApprovals();
  if (!approvalsResult.ok) notFound();

  const approval = approvalsResult.data.find((item) => item.id === id);
  if (!approval) notFound();

  const recommendation = await getRecommendationById(approval.recommendationId);
  if (!recommendation.ok) notFound();

  return (
    <div className="space-y-6 p-6">
      <PageHeader title={approval.title} description="Approval workspace with
evidence, savings, risk, and rollback context." />
      <div className="grid gap-6 xl:grid-cols-[1.2fr_.8fr]">
        <Card className="rounded-3xl border-0 shadow-sm">
          <CardHeader><CardTitle>Evidence</CardTitle></CardHeader>
          <CardContent className="space-y-3 text-sm text-slate-600">
            {recommendation.data.evidence.map((line) => <div key={line}>• {line}
</div>)}
```

```
          </CardContent>
        </Card>
        <Card className="rounded-3xl border-0 shadow-sm">
          <CardHeader><CardTitle>Decision Notes</CardTitle></CardHeader>
          <CardContent className="space-y-4">
            <Textarea defaultValue="Proceed after notifying impacted owners."
className="min-h-[180px]" />
            <div className="grid gap-3 sm:grid-cols-2">
              <Button>Approve</Button>
              <Button variant="destructive">Reject</Button>
              <Button variant="outline">Defer</Button>
              <Button variant="outline">Escalate</Button>
            </div>
          </CardContent>
        </Card>
      </div>
    </div>
  );
}
```

src/app/(dashboard)/executions/[id]/page.tsx

```
import { notFound } from "next/navigation";
import { PageHeader } from "@/components/shared/page-header";
import { requirePermission } from "@/lib/auth/guards";
import { listExecutions } from "@/lib/mock/repositories/executions-repo";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";

export default async function ExecutionDetailPage({ params }: { params:
Promise<{ id: string }> }) {
  await requirePermission("executeActions");
  const { id } = await params;

  const result = await listExecutions();
  if (!result.ok) notFound();

  const execution = result.data.find((item) => item.id === id);
  if (!execution) notFound();

  return (
    <div className="space-y-6 p-6">
      <PageHeader title={execution.recommendationTitle} description={`$
{execution.id} · ${execution.mode}`} />
      <Card className="rounded-3xl border-0 shadow-sm">
        <CardHeader>
```

```
          <CardTitle>Execution Plan</CardTitle>
        </CardHeader>
        <CardContent className="space-y-3">
          {execution.steps.map((step) => (
            <div key={step.label} className="flex items-center justify-between
rounded-2xl border p-4 text-sm">
              <span>{step.label}</span>
              <span>{step.done ? "Done" : "Pending"}</span>
            </div>
          ))}
          <div className="flex flex-wrap gap-3 pt-2">
            <Button>Pause</Button>
            <Button variant="outline">View Logs</Button>
            <Button variant="destructive">Rollback</Button>
          </div>
        </CardContent>
      </Card>
    </div>
  );
}
```

What Batch 3 completes

This batch completes the main missing structure from the frontend scaffold: - governance/admin pages - approval and execution detail routes - starter mutations for decisions and rollback actions - typed settings and policy data - reusable loading/error wrappers - tenant and role switcher components

Best next batch

Batch 4 should add: - real route-level loading and error files - breadcrumbs - search/filter state hooks - server action or API route examples - TanStack Table versions of approvals/executions/renewals - test scaffolds for permissions and route guards - backend scaffold matching these modules